

СПИСЪК ОТ КЛАСОВЕ – вградени възможности

Като продължение от предишния урок да тестваме вмъкване, отстраняване и други вградени свойства и методи, защото би било доста по-бързо и лесно да използваме вградено property `.Average`, отколкото да изчислим средна стойност като намерим сумата на елементите, преброим ги и разделим. Има само няколко особености, които е добре да споменем:

- 1) Когато тестваме вградена функционалност на списъка, изписваме неговото име в (малка буква, мн.ч. `teachers`) и след това наследяването **точка** . – избираме например `Exists()`, който, както показва името му, връща `true/false` дали нещо съществува в списъка. Дори да не познаваме даден вграден метод, се ориентираме какъв е неговият правилен синтаксис, като прочетем подсказката. Думата **"Predicate"** веднага трябва да ни говори за знака `=>` и съответно конструкция, която сочи към член на дадения списък като `t => t`, където няма значение каква буква изберем, може да бъде всяка, но се предпочита за по-кратко първата буква от името на списъка. Други предпочитани са `x => x` (неизвестно) или `e => e` (елемент). Макар и неофициално, съм ви сравнявала знака `=>` в този случай като търсачка, защото сочи към обект и търси съвпадение, а кое точно – избираме следващото property чрез наследяване с точка . – в нашия случай `.Age`. Следващата важна стъпка от разчитане на подсказката е думата **match**, която говори за знаците за съпадение/сравнение `<, <=, >, >=, ==` или понякога дори `.Contains`.

```
teachers.Exists(t => t.Age < 25);  
// Console.Writ  
Teacher found
```

`bool List<Teacher>.Exists(Predicate<Teacher> match)`
Determines whether the `List<T>` contains elements that match the conditions defined by the specified predicate.

- 2) Вторият основен момент е, че нашата конструкция дори в този си вид е готова и може да работи правилно, няма никакви синтактични грешки, съответно може да бъде вложена в `CW` директно. Ако я оставим извън `CW`, тя ще е изпълнила действието, за което е предназначена, но няма да бъде показана или запазена по някакъв начин, затова можем да изместим курсора пред нея и присвоим резултата от изпълнението ѝ в нова променлива, която трябва да бъде тип `bool`. Просто я именуваме.

```
bool exists = teachers.Exists(t => t.Age < 25);  
Console.WriteLine("Teachers under 25: " + exists);
```

Желателно е булевите променливи да започват с `is` в името си, но в случая предпочетох да бъде **exists**, вместо `isExisting`.

Още малко пояснения за разликите между `.Find()` и `.FindAll()`

```
Teacher found = teachers.Find(t => t.Name == "Aleksandrov");  
found.DisplayInfo();
```

`Teacher? List<Teacher>.Find(Predicate<Teacher> match)` `.Find()` връща 1 ел - първия съвпадащ
Searches for an element that matches the conditions defined by the specified predicate, and returns the first occurrence within the entire `List<T>`.

.Find() връща 1 елемент от списъка (първият, който съвпада), затова той е сложен-съставен тип `Teacher`.

Ако прочетем надолу, в частта на подсказката Returns: пише директно, че връща първият елемент, съответно само едно отпечатване с CW е достатъчно.

```
Teacher found = teachers.Find(t => t.Name == "Aleksandrov");
found.DisplayInfo();
```

Teacher? List<Teacher>.Find(Predicate<Teacher> match)
Searches for an element that matches the conditions defined by the specified predicate, and returns the first occurrence within the entire List<T>.

Returns:
The first element that matches the conditions defined by the specified predicate, if found; otherwise, the default value for type T.

Като съществуващ елемент от списък, за него може да бъде извикан и публичния метод:

```
Teacher found = teachers.Find(t => t.Name == "Aleksandrov");
found.DisplayInfo(); // .Find = връща първия намерен елемент
```

Когато използваме другата възможност за търсене **FindAll()**, както показва името ѝ, тя ще връща като резултат всички елементи, които отговарят на критерия за съвпадение.

```
List<Teacher> aleks = teachers.FindAll(t => t.Name == "Aleksandrov");
foreach (Teacher a in aleks) //
{
    Console.WriteLine($"Found by {a.Name}");
}
```

List<Teacher> List<Teacher>.FindAll(Predicate<Teacher> match)
Retrieves all the elements that match the conditions defined by the specified predicate.

Returns:
A List<T> containing all the elements that match the conditions defined by the specified predicate.

Затова:

- 1) резултатът не е 1 елемент, а цял списък и като такъв, трябва да бъде указан тип List <Teacher> - има го направо в подсказката.
- 2) За отпечатване няма да е достатъчен 1 CW, а направо обхождане с foreach.
- 3) Листът aleks (от резултата) се явява подписък на teachers, но е и самостоятелна единица вече.

Другото, което може да отбележим е, че може изпълнението да бъде в 1-2-3 стъпки, както е показано (със запазване на резултата като променлива):

```
int maxAge = teachers.Max(t => t.Age);
Console.WriteLine($"Max age: {maxAge:F1} years."); // по-гъвкав запис
Console.WriteLine($"Max age shorten syntax: {teachers.Max(t => t.Age):F2}"); // директно
```

или директно в CW, ако няма да ни трябва за друго нещо по-надолу в кода.

Друга забележка: **Add()** добавя нов елемент (на последно място), **Insert()** в коя точно позиция (от 0 до .Count-1) да се вмъкне този елемент, а **AddRange()** вмъква цял готов списък:

```
teachers.AddRange(sorted); // добавяме втори списък към първия
```

И разбира се, може да използваме и по няколко наследени вградени метода последователно:

```
List<Teacher> age25 = teachers.Where(t => t.Age > 25).ToList();
```

Ето пълния код от упражнението:

```
List<Teacher> teachers = new List<Teacher>();

teachers.Add(new Teacher( "Ivanov", 40 ));
teachers.Add(new Teacher("Georgieva", 32));
teachers.Add(new Teacher("Dimitrova", 55));

foreach (Teacher teacher in teachers)
{
    teacher.DisplayInfo();
}

teachers[0].DisplayInfo(); // първи
teachers[teachers.Count - 1].DisplayInfo(); // последен

teachers.Add(new Teacher("Vasileva", 25));
teachers.Add(new Teacher("Aleksandrov", 60));

teachers.Insert(3, new Teacher("Evgenieva", 57));
teachers.Insert(0, new Teacher("Stamov", 44));

teachers.RemoveAt(0);
//Teacher toRemove = teachers[0];
//teachers.Remove(toRemove);

bool exists = teachers.Exists(t => t.Age < 25);
Console.WriteLine("Teachers under 25: " + exists);

Teacher found = teachers.Find( t => t.Name == "Aleksandrov");
found.DisplayInfo(); // .Find = връща първия намерен елемент
```

```

double avgAge = teachers.Average(t => t.Age);
Console.WriteLine($"Average age: {avgAge:F1} years.");

int maxAge = teachers.Max(t => t.Age);
Console.WriteLine($"Max age: {maxAge:F1} years."); // по-дълъг запис
Console.WriteLine($"Max age shorten syntax: {teachers.Max(t => t.Age):F2}"); //директно

teachers.Add(new Teacher("Aleksandrov", 23));

List<Teacher> aleks = teachers.FindAll(t => t.Name == "Aleksandrov");
foreach (Teacher a in aleks) // .FindAll = връща всички срещнати
{
    Console.WriteLine($"Found by name: {a.Name}, {a.Age}");
}

List<Teacher> age25 = teachers.Where(t => t.Age > 25).ToList();
foreach (Teacher a in age25)
{
    Console.WriteLine($"Found by age: {a.Name}, {a.Age}");
}

List<Teacher> sorted = teachers.OrderBy(t => t.Name).ToList(); // по азбучен ред

foreach (Teacher teacher in sorted)
{
    Console.WriteLine(teacher.Name);
}

teachers.AddRange(sorted); // добавяме втори списък към първия

        foreach (Teacher teacher in teachers)
        {
            teacher.DisplayInfo();
        }

```

```

List<Teacher> teachers = new List<Teacher>();

teachers.Add(new Teacher( "Ivanov", 40 ));
teachers.Add(new Teacher("Georgieva", 32));
teachers.Add(new Teacher("Dimitrova", 55));

foreach (Teacher in teachers)
{
    teacher.DisplayInfo();
}

teachers[0].DisplayInfo(); // първи
teachers[teachers.Count - 1].DisplayInfo(); // последен

teachers.Add(new Teacher("Vasileva", 25));
teachers.Add(new Teacher("Aleksandrov", 60));

teachers.Insert(3, new Teacher("Evgenieva", 57));
teachers.Insert(0, new Teacher("Stamov", 44));

```

```

teachers.RemoveAt(0);
//Teacher toRemove = teachers[0];
//teachers.Remove(toRemove);

bool exists = teachers.Exists(t => t.Age < 25);
Console.WriteLine("Teachers under 25: " + exists);

Teacher found = teachers.Find( t => t.Name == "Aleksandrov");
found.DisplayInfo(); // .Find = връща първия намерен елемент

double avgAge = teachers.Average(t => t.Age);
Console.WriteLine($"Average age: {avgAge:F1} years.");

int maxAge = teachers.Max(t => t.Age);
Console.WriteLine($"Max age: {maxAge:F1} years."); // по-дълъг запис
Console.WriteLine($"Max age shorten syntax: {teachers.Max(t =>
t.Age):F2}"); //директно

teachers.Add(new Teacher("Aleksandrov", 23));

List<Teacher> aleks = teachers.FindAll(t => t.Name == "Aleksandrov");
foreach (Teacher a in aleks) // .FindAll = връща всички срещнати
{
    Console.WriteLine($"Found by name: {a.Name}, {a.Age}");
}

List<Teacher> age25 = teachers.Where(t => t.Age > 25).ToList();
foreach (Teacher a in age25)
{
    Console.WriteLine($"Found by age: {a.Name}, {a.Age}");
}

List<Teacher> sorted = teachers.OrderBy(t => t.Name).ToList(); // по
азбучен ред

foreach (Teacher teacher in sorted)
{
    Console.WriteLine(teacher.Name);
}

teachers.AddRange(sorted); // добавяме втори списък към първия

foreach (Teacher teacher in teachers)
{
    teacher.DisplayInfo();
}

```